

React.js

Complete Learning Material

From Scratch to Associate Software Engineer

Java Full Stack Developer — L1 & L2 Interview Preparation

Topic	Coverage
Core React	JSX, Components, Props, State, Events
Hooks	useState, useEffect, useContext, useReducer, etc.
Routing	React Router v6
State Management	Context API, Redux Toolkit
API Integration	Fetch, Axios, REST with Spring Boot
Advanced	HOCs, Error Boundaries, Performance
Testing	Jest, React Testing Library basics
Interview Prep	100+ L1 & L2 Q&A

Table of Contents

- 1. Introduction to React
- 2. Setting Up the Environment
- 3. JSX — JavaScript XML
- 4. Components: Functional & Class
- 5. Props — Passing Data
- 6. State Management Basics
- 7. Event Handling
- 8. Conditional Rendering
- 9. Lists and Keys
- 10. Forms: Controlled & Uncontrolled
- 11. Component Lifecycle
- 12. React Hooks — Deep Dive
- 13. Custom Hooks
- 14. Context API
- 15. React Router
- 16. Redux & Redux Toolkit
- 17. API Integration (Fetch / Axios)
- 18. Error Boundaries
- 19. Higher-Order Components & Render Props
- 20. Performance Optimization
- 21. Testing React Apps
- 22. Integrating React with Spring Boot
- 23. Best Practices & Folder Structure
- 24. L1 Interview Questions (50+)
- 25. L2 Interview Questions (50+)
- 26. Coding Round Problems
- 27. Quick Revision Cheat Sheet

1. Introduction to React

React is an open-source JavaScript library developed by Facebook (now Meta) in 2013 for building user interfaces, particularly single-page applications (SPAs). It allows developers to create reusable UI components that manage their own state, leading to predictable and efficient rendering.

Why React?

- **Component-Based:** UI is broken down into small, reusable pieces.
- **Virtual DOM:** Efficient diffing algorithm updates only what changed.
- **Declarative:** Describe *what* the UI should look like, not *how* to update it.
- **Unidirectional Data Flow:** Data flows top-down (parent to child).
- **Rich Ecosystem:** React Router, Redux, Next.js, Material UI, etc.
- **Strong Community:** Maintained by Meta with millions of developers.

React vs Other Frameworks

Feature	React	Angular	Vue
Type	Library	Framework	Framework
Language	JavaScript/JSX	TypeScript	JavaScript
DOM	Virtual DOM	Real DOM + Change Detection	Virtual DOM
Data Binding	One-way	Two-way	Two-way
Learning Curve	Moderate	Steep	Easy

Key Concepts to Master

- **Virtual DOM:** An in-memory representation of the real DOM. React compares the new VDOM with the previous one (diffing) and updates only the changed parts (reconciliation).
- **JSX:** Syntax extension that lets you write HTML inside JavaScript.
- **Components:** Independent, reusable pieces of UI.
- **Props & State:** Mechanisms to pass data and manage internal data.
- **Hooks:** Functions that let functional components use state and lifecycle features.

2. Setting Up the Environment

Prerequisites

- Node.js (v18 or higher) — JavaScript runtime
- npm or yarn — Package managers
- Code editor (VS Code recommended)
- Basic HTML, CSS, JavaScript (ES6+) knowledge

Creating a React App

There are two popular ways: **Create React App (CRA)** — legacy but simple, or **Vite** — modern, faster build tool. Vite is recommended in 2025.

Using Vite (Recommended)

```
# Create a new React app with Vite
npm create vite@latest my-app -- --template react

cd my-app
npm install
npm run dev

# Opens at http://localhost:5173
```

Using Create React App (Legacy)

```
npx create-react-app my-app
cd my-app
npm start

# Opens at http://localhost:3000
```

Project Structure

```
my-app/
├── node_modules/  # Dependencies
├── public/        # Static files (index.html, favicon)
├── src/          # Source code
│   ├── components/ # Reusable components
│   ├── pages/      # Page-level components
│   ├── hooks/      # Custom hooks
│   ├── services/   # API calls
│   ├── utils/      # Helper functions
│   ├── App.jsx     # Root component
│   ├── main.jsx    # Entry point
│   └── index.css   # Global styles
├── package.json  # Project metadata & dependencies
└── vite.config.js # Vite configuration
```

Essential Commands

```
npm install <package>  # Install a package
npm install             # Install all dependencies
npm run dev             # Start dev server (Vite)
npm run build           # Production build
npm run preview         # Preview production build
npm test               # Run tests
```

3. JSX — JavaScript XML

JSX is a syntax extension for JavaScript that looks similar to HTML. It allows you to write HTML-like markup inside JavaScript code. Behind the scenes, JSX is transpiled to `React.createElement()` calls.

JSX vs HTML

```
// JSX
const element = <h1 className="title">Hello, World!</h1>;

// Equivalent JavaScript (what JSX compiles to)
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello, World!'
);
```

JSX Rules

- Must return a single parent element (use Fragments `<></>` for multiple)
- All tags must be closed: `<img />`, `<br />`
- Use `className` instead of `class`
- Use `htmlFor` instead of `for`
- camelCase for attributes: `onClick`, `tabIndex`
- JavaScript expressions inside curly braces `{ }`

Embedding Expressions

```
const name = "Alice";
const age = 25;

const greeting = (
  <div>
    <h1>Hello, {name}!</h1>
    <p>You are {age} years old.</p>
    <p>Next year: {age + 1}</p>
    <p>{age >= 18 ? 'Adult' : 'Minor'}</p>
  </div>
);
```

Fragments

Fragments let you group elements without adding extra DOM nodes.

```
// Using <> shorthand
function List() {
  return (
    <>
      <li>Item 1</li>
      <li>Item 2</li>
    </>
  );
}

// Using React.Fragment (needed when you want to pass a key)
<React.Fragment key={id}>...</React.Fragment>
```

Inline Styles

```
// Style as JavaScript object - camelCase properties
const styles = {
  color: 'red',
  backgroundColor: 'yellow',
  fontSize: '16px'
};

<div style={styles}>Styled text</div>
<div style={{ padding: 10, margin: 5 }}>Inline</div>
```

Note: JSX is NOT mandatory in React — you can use `React.createElement()` directly — but JSX is far cleaner and standard practice.

4. Components: Functional & Class

Components are the building blocks of React applications. A component is essentially a JavaScript function or class that returns JSX describing what should appear on the screen.

Functional Components (Modern Approach)

```
// Simple functional component
function Welcome() {
  return <h1>Hello, World!</h1>;
}

// Arrow function syntax (preferred)
const Welcome = () => {
  return <h1>Hello, World!</h1>;
};

// Implicit return for single expression
const Welcome = () => <h1>Hello, World!</h1>;

// Using the component
<Welcome />
```

Class Components (Legacy)

```
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h1>Hello, World!</h1>;
  }
}

export default Welcome;
```

Functional vs Class Components

Aspect	Functional	Class
Syntax	Function/Arrow	ES6 Class
State	useState hook	this.state
Lifecycle	useEffect hook	Lifecycle methods
this keyword	Not used	Required
Performance	Slightly faster	Slightly slower
Modern Standard	Yes (recommended)	Legacy

Component Naming Convention

- **PascalCase** for component names: MyComponent, not myComponent
- React treats lowercase tags as HTML elements
- File names typically match component names: MyComponent.jsx

Composing Components

```
function Header() {
```

```
    return <h1>My App</h1>;
  }

  function Footer() {
    return <p>&copy; 2025</p>;
  }

  function App() {
    return (
      <div>
        <Header />
        <main>Welcome!</main>
        <Footer />
      </div>
    );
  }
}
```


5. Props — Passing Data

Props (short for properties) are how data is passed from a parent component to a child component. Props are **read-only** — a component must never modify its own props.

Passing & Receiving Props

```
// Parent component
function App() {
  return <Greeting name="Alice" age={25} isAdmin={true} />;
}

// Child component - receiving props
function Greeting(props) {
  return (
    <div>
      <h1>Hello, {props.name}</h1>
      <p>Age: {props.age}</p>
      {props.isAdmin && <p>Admin user</p>}
    </div>
  );
}

// Destructuring (preferred)
function Greeting({ name, age, isAdmin }) {
  return <h1>Hello, {name}</h1>;
}
```

Default Props

```
// Using destructuring with defaults
function Button({ label = 'Click', color = 'blue' }) {
  return <button style={{ color }}>{label}</button>;
}

// Old way: defaultProps (still works but discouraged)
Button.defaultProps = {
  label: 'Click',
  color: 'blue'
};
```

Children Prop

Special children prop represents content between the opening and closing tags of a component.

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage
<Card title="Welcome">
  <p>This is the content</p>
  <button>Click me</button>
</Card>
```

Prop Types (Type Checking)

```
import PropTypes from 'prop-types';

function User({ name, age, email }) {
  return <div>{name}, {age}, {email}</div>;
}

User.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  email: PropTypes.string,
  hobbies: PropTypes.arrayOf(PropTypes.string),
  address: PropTypes.shape({
    city: PropTypes.string,
    zip: PropTypes.string
  })
};
```

Passing Functions as Props

Functions can be passed as props to enable child-to-parent communication.

```
function Parent() {
  const handleClick = (msg) => alert(msg);
  return <Child onClick={handleClick} />;
}

function Child({ onClick }) {
  return <button onClick={() => onClick('Hi from child!')}>Click</button>;
}
```

Note: Props are immutable! Never modify them inside a child component. If you need to change data, use state in the parent and pass a setter function.

6. State Management Basics

State represents data that can change over time within a component. When state changes, React re-renders the component to reflect the new data.

useState Hook

```
import { useState } from 'react';

function Counter() {
  // useState returns: [currentValue, setterFunction]
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

State with Different Data Types

```
// String state
const [name, setName] = useState('');

// Number state
const [age, setAge] = useState(0);

// Boolean state
const [isOpen, setIsOpen] = useState(false);

// Array state
const [items, setItems] = useState([]);
setItems([...items, newItem]); // Add item (immutable)

// Object state
const [user, setUser] = useState({ name: '', email: '' });
setUser({ ...user, name: 'Alice' }); // Update one field
```

Functional Updates

When new state depends on previous state, use the functional form to avoid stale state issues.

```
// WRONG - uses possibly stale count
setCount(count + 1);
setCount(count + 1); // Both reference the SAME old count!

// CORRECT - uses functional form
setCount(prev => prev + 1);
setCount(prev => prev + 1); // Both increments will work
```

State Update is Asynchronous

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(count + 1);
  console.log(count); // Still prints OLD value!
```

```
    // To see new value, use useEffect
  }
```

Lifting State Up

When two child components need to share state, move (lift) the state up to their common parent.

```
function Parent() {
  const [shared, setShared] = useState('');
  return (
    <>
      <ChildA value={shared} onChange={setShared} />
      <ChildB value={shared} />
    </>
  );
}
```

Note: State updates are batched in React 18+. Multiple state updates in the same event handler are processed together for performance.

7. Event Handling

React events use camelCase naming and pass a function (not a string) as the handler. React wraps native events in a **SyntheticEvent** for cross-browser consistency.

Common Event Handlers

```
function App() {
  const handleClick = () => console.log('Clicked!');
  const handleChange = (e) => console.log(e.target.value);
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Form submitted');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} />
      <button onClick={handleClick}>Click</button>
    </form>
  );
}
```

Passing Arguments

```
// Method 1: Inline arrow function
<button onClick={() => deleteItem(id)}>Delete</button>

// Method 2: Bind (less common)
<button onClick={deleteItem.bind(null, id)}>Delete</button>

// Method 3: Higher-order function
const handleDelete = (id) => () => deleteItem(id);
<button onClick={handleDelete(id)}>Delete</button>
```

Common Synthetic Events

Event Type	Handler	Example
Mouse	onClick, onMouseEnter, onMouseLeave	Buttons, hover
Keyboard	onKeyDown, onKeyUp, onKeyPress	Input fields
Form	onChange, onSubmit, onFocus, onBlur	Forms
Touch	onTouchStart, onTouchEnd	Mobile gestures
Scroll	onScroll	Scroll containers

preventDefault & stopPropagation

```
function Form() {
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    // process form
  };

  const handleInnerClick = (e) => {
    e.stopPropagation(); // Stop event bubbling
  };

  return (
```

```
    <div onClick={() => console.log('outer')}>
      <button onClick={handleInnerClick}>Inner</button>
    </div>
  );
}
```

8. Conditional Rendering

Render different UI based on conditions using JavaScript operators.

If-Else with Variables

```
function Greeting({ isLoggedIn }) {
  let message;
  if (isLoggedIn) {
    message = <h1>Welcome back!</h1>;
  } else {
    message = <h1>Please log in.</h1>;
  }
  return <div>{message}</div>;
}
```

Ternary Operator

```
function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <h1>Welcome!</h1> : <h1>Please log in</h1>}
    </div>
  );
}
```

Logical AND (&&)

```
function Notifications({ count }) {
  return (
    <div>
      {count > 0 && <span>You have {count} new messages</span>}
    </div>
  );
}
```

// CAUTION: 0 will render as "0", not nothing
// Use {count > 0 && ...} not {count && ...}

Switch-Case Pattern

```
function Status({ type }) {
  const components = {
    loading: <Spinner />,
    error: <ErrorMessage />,
    success: <SuccessView />
  };
  return components[type] || <DefaultView />;
}
```

Returning null

```
function Warning({ show }) {
  if (!show) return null; // Component renders nothing
  return <div className="warning">Watch out!</div>;
}
```

9. Lists and Keys

Use JavaScript's `.map()` to transform arrays into lists of JSX elements. Each item must have a unique `key` prop.

Rendering a List

```
function FruitList() {
  const fruits = ['Apple', 'Banana', 'Cherry'];
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Rendering List of Objects

```
const users = [
  { id: 1, name: 'Alice', email: 'alice@test.com' },
  { id: 2, name: 'Bob', email: 'bob@test.com' }
];

function UserList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          {user.name} - {user.email}
        </li>
      ))}
    </ul>
  );
}
```

Why Keys Matter

- Keys help React identify which items changed, added, or removed
- Improve performance of re-renders via the reconciliation algorithm
- Should be **unique** among siblings
- Should be **stable** (don't generate random keys on every render)
- Avoid using array index as key if list can be reordered/filtered

Bad vs Good Keys

```
// BAD - using index for dynamic lists
{items.map((item, i) => <li key={i}>{item.text}</li>)}

// BAD - using Math.random()
{items.map(item => <li key={Math.random()}>{item}</li>)}

// GOOD - using stable unique ID
{items.map(item => <li key={item.id}>{item.text}</li>)}
```

Filtering & Sorting


```
const activeUsers = users
  .filter(u => u.active)
  .sort((a, b) => a.name.localeCompare(b.name));

return (
  <ul>
    {activeUsers.map(u => <li key={u.id}>{u.name}</li>)}
  </ul>
);
```

10. Forms: Controlled & Uncontrolled

Controlled Components

Form data is controlled by React state. The input's value is set by state, and changes go through a handler.

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log({ email, password });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

Multiple Inputs - Single Handler

```
function Form() {
  const [form, setForm] = useState({
    name: '', email: '', age: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  };

  return (
    <form>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <input name="age" value={form.age} onChange={handleChange} />
    </form>
  );
}
```

Uncontrolled Components (with useRef)

```
import { useRef } from 'react';

function Form() {
  const inputRef = useRef(null);
```

```

const handleSubmit = (e) => {
  e.preventDefault();
  console.log(inputRef.current.value);
};

return (
  <form onSubmit={handleSubmit}>
    <input ref={inputRef} defaultValue="Hello" />
    <button>Submit</button>
  </form>
);
}

```

Form Validation

```

function SignupForm() {
  const [email, setEmail] = useState('');
  const [errors, setErrors] = useState({});

  const validate = () => {
    const errs = {};
    if (!email) errs.email = 'Email required';
    else if (!/\S+@\S+\.\S+/.test(email))
      errs.email = 'Invalid email';
    return errs;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    if (Object.keys(errs).length === 0) {
      // submit data
    } else {
      setErrors(errs);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      {errors.email && <span style={{color:'red'}}>{errors.email}</span>}
      <button>Sign Up</button>
    </form>
  );
}

```

Controlled vs Uncontrolled

Aspect	Controlled	Uncontrolled
Source of truth	React state	DOM
Value access	via state	via ref
Validation	Easy (on change)	Harder
Performance	Re-renders on each keystroke	No re-renders
Preferred for	Most use-cases	Simple/legacy forms

11. Component Lifecycle

Lifecycle refers to the stages a component goes through: mounting, updating, and unmounting. Class components use lifecycle methods; functional components use the `useEffect` hook.

Three Phases

- **Mounting:** Component is created and inserted into the DOM.
- **Updating:** Component re-renders due to state or prop changes.
- **Unmounting:** Component is removed from the DOM.

Class Component Lifecycle Methods

Phase	Method	When Called
Mount	constructor()	Before mounting; init state
Mount	render()	Returns JSX
Mount	componentDidMount()	After first render; API calls
Update	shouldComponentUpdate()	Before re-render (perf)
Update	render()	On state/prop change
Update	componentDidUpdate()	After re-render
Unmount	componentWillUnmount()	Before removal; cleanup

Class Lifecycle Example

```
class Timer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  componentDidMount() {
    this.timer = setInterval(() => {
      this.setState({ count: this.state.count + 1 });
    }, 1000);
  }

  componentDidUpdate(prevProps, prevState) {
    if (this.state.count !== prevState.count) {
      console.log('Count changed');
    }
  }

  componentWillUnmount() {
    clearInterval(this.timer); // Cleanup
  }

  render() {
    return <h1>{this.state.count}</h1>;
  }
}
```

Functional Component Equivalent

```

function Timer() {
  const [count, setCount] = useState(0);

  // componentDidMount + componentWillUnmount
  useEffect(() => {
    const timer = setInterval(() => {
      setCount(prev => prev + 1);
    }, 1000);
    return () => clearInterval(timer); // Cleanup
  }, []);

  // componentDidUpdate (for specific value)
  useEffect(() => {
    console.log('Count changed');
  }, [count]);

  return <h1>{count}</h1>;
}

```

12. React Hooks — Deep Dive

Hooks are functions that let functional components use state and other React features. Introduced in React 16.8.

Rules of Hooks

- Only call hooks at the **top level** of a function (not inside loops, conditions, or nested functions)
- Only call hooks from React function components or other custom hooks
- Hook names must start with use

useState

```
const [count, setCount] = useState(0);
const [user, setUser] = useState({ name: '' });

// Lazy initialization (for expensive computations)
const [data, setData] = useState(() => computeInitialData());
```

useEffect

Runs side effects: data fetching, subscriptions, manually changing DOM, timers, etc.

```
// Run after every render
useEffect(() => {
  console.log('Rendered');
});

// Run only ONCE (on mount)
useEffect(() => {
  fetchData();
}, []);

// Run when 'userId' changes
useEffect(() => {
  fetchUser(userId);
}, [userId]);

// With cleanup (on unmount or before re-run)
useEffect(() => {
  const sub = subscribe();
  return () => sub.unsubscribe();
}, [topic]);
```

useContext

Access values from React Context without prop drilling.

```
const ThemeContext = createContext('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
```

```

const theme = useContext(ThemeContext);
return <div className={theme}>Toolbar</div>;
}

```

useReducer

Alternative to useState for complex state logic. Similar to Redux.

```

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    case 'reset': return initialState;
    default: throw new Error();
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <>
      <p>{state.count}</p>
      <button onClick={() => dispatch({type: 'increment'})}>+</button>
      <button onClick={() => dispatch({type: 'decrement'})}>-</button>
    </>
  );
}

```

useRef

Returns a mutable ref object. Common uses: accessing DOM nodes, storing values that don't trigger re-renders.

```

// Access DOM element
function TextInput() {
  const inputRef = useRef(null);
  useEffect(() => {
    inputRef.current.focus(); // Auto-focus on mount
  }, []);
  return <input ref={inputRef} />;
}

// Store mutable value (no re-render)
function Timer() {
  const intervalRef = useRef(null);
  const start = () => {
    intervalRef.current = setInterval(() => { /* ... */ }, 1000);
  };
  const stop = () => clearInterval(intervalRef.current);
}

```

useMemo

Memoizes the result of an expensive computation. Recomputes only when dependencies change.

```

function ExpensiveList({ items, filter }) {
  const filtered = useMemo(() => {
    return items.filter(i => i.name.includes(filter));
  }, [items, filter]);

  return <ul>{filtered.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
}

```

```
}
```

useCallback

Memoizes a function reference. Useful when passing callbacks to optimized child components.

```
const handleClick = useCallback(() => {  
  doSomething(a, b);  
}, [a, b]);  
  
// Without useCallback, a new function is created on every render,  
// causing memoized child components to re-render unnecessarily
```

Other Hooks

- **useLayoutEffect:** Like `useEffect` but fires synchronously after DOM mutations
- **useImperativeHandle:** Customize the instance value exposed when using `ref`
- **useDebugValue:** Display custom label for hooks in React DevTools
- **useId:** Generate unique IDs for accessibility (React 18+)
- **useTransition / useDeferredValue:** Mark state updates as non-urgent (React 18+)

13. Custom Hooks

Custom hooks let you extract component logic into reusable functions. Their name must start with use.

Example: useToggle

```
function useToggle(initial = false) {
  const [value, setValue] = useState(initial);
  const toggle = useCallback(() => setValue(v => !v), []);
  return [value, toggle];
}

// Usage
function App() {
  const [isOpen, toggleOpen] = useToggle(false);
  return <button onClick={toggleOpen}>{isOpen ? 'Close' : 'Open'}</button>;
}
```

Example: useFetch

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => res.json())
      .then(setData)
      .catch(setError)
      .finally(() => setLoading(false));
  }, [url]);

  return { data, loading, error };
}

// Usage
function Users() {
  const { data, loading, error } = useFetch('/api/users');
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Example: useLocalStorage

```
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage
const [theme, setTheme] = useLocalStorage('theme', 'light');
```

Example: useDebounce

```
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const handler = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(handler);
  }, [value, delay]);
  return debounced;
}

// Usage: search input that only triggers API after 500ms idle
function Search() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 500);
  useEffect(() => {
    if (debouncedQuery) fetchResults(debouncedQuery);
  }, [debouncedQuery]);
}
```

14. Context API

Context provides a way to pass data through the component tree without prop drilling. Useful for global data like themes, user authentication, language settings.

Creating & Using Context

```
// 1. Create context
import { createContext, useContext, useState } from 'react';
const AuthContext = createContext(null);

// 2. Provider component (wraps your app)
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => setUser(userData);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// 3. Custom hook (optional but recommended)
export const useAuth = () => useContext(AuthContext);

// 4. Use in app
function App() {
  return (
    <AuthProvider>
      <MainPage />
    </AuthProvider>
  );
}

function Profile() {
  const { user, logout } = useAuth();
  return <div>Hi {user.name}! <button onClick={logout}>Logout</button></div>;
}
```

When to Use Context

- Theme (dark/light mode)
- User authentication state
- Locale/language
- App settings
- Any global, infrequently-changing data

Note: Context causes ALL consumers to re-render when the value changes. For frequently-changing state, use Redux/Zustand or split into smaller contexts.

15. React Router

React Router enables navigation between different views in a SPA without full page reloads. Current major version: v6+.

Installation & Setup

```
npm install react-router-dom
```

Basic Routing

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users">Users</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users" element={<Users />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Route Parameters

```
import { useParams } from 'react-router-dom';

// Define route with parameter
<Route path="/users/:id" element={<UserDetail />} />

// Read parameter
function UserDetail() {
  const { id } = useParams();
  return <h1>User ID: {id}</h1>;
}
```

Programmatic Navigation

```
import { useNavigate } from 'react-router-dom';

function Login() {
  const navigate = useNavigate();

  const handleSubmit = async () => {
    const ok = await loginUser();
    if (ok) navigate('/dashboard');
  };
}
```

Query Parameters

```
import { useSearchParams } from 'react-router-dom';
```

```
function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get('category');
  const page = searchParams.get('page') || '1';

  return (
    <div>
      <p>Showing {category}, page {page}</p>
      <button onClick={() => setSearchParams({category, page: '2'})}>
        Next
      </button>
    </div>
  );
}
```

Nested Routes

```
import { Outlet } from 'react-router-dom';

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <main><Outlet /></main>    {/* Child route renders here */}
    </div>
  );
}

<Routes>
  <Route path="/dashboard" element={<DashboardLayout />}>
    <Route index element={<Overview />} />
    <Route path="profile" element={<Profile />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Routes>
```

Protected Routes

```
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

<Route path="/admin" element={
  <ProtectedRoute><Admin /></ProtectedRoute>
} />
```

16. Redux & Redux Toolkit

Redux is a predictable state container for managing application state. **Redux Toolkit (RTK)** is the official, modern way to write Redux logic with less boilerplate.

Core Concepts

- **Store:** Single source of truth holding the application state
- **Action:** Plain object describing what happened ({type, payload})
- **Reducer:** Pure function (state, action) => newState
- **Dispatch:** Method to send actions to the store
- **Selector:** Function to extract specific data from state

Redux Data Flow

```
User Event -> dispatch(action) -> reducer(state, action)
                                     -> store updated -> UI re-renders
```

Setup with Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
```

Create a Slice

```
// features/counter/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1 },
    decrement: (state) => { state.value -= 1 },
    incrementBy: (state, action) => {
      state.value += action.payload;
    }
  }
});

export const { increment, decrement, incrementBy } = counterSlice.actions;
export default counterSlice.reducer;
```

Configure the Store

```
// app/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../features/counter/counterSlice';
import userReducer from '../features/user/userSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer
  }
});
```

Provide Store to App

```
// main.jsx
import { Provider } from 'react-redux';
import { store } from './app/store';

<Provider store={store}>
  <App />
</Provider>
```

Use in Components

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from './counterSlice';

function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
    </div>
  );
}
```

Async Logic with createAsyncThunk

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';

export const fetchUsers = createAsyncThunk(
  'users/fetch',
  async () => {
    const res = await fetch('/api/users');
    return res.json();
  }
);

const userSlice = createSlice({
  name: 'users',
  initialState: { list: [], status: 'idle' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.status = 'loading' })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = 'success';
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state) => { state.status = 'failed' });
  }
});
```

17. API Integration (Fetch / Axios)

Using Fetch API

```
useEffect(() => {
  fetch('https://api.example.com/users')
    .then(res => {
      if (!res.ok) throw new Error('Network error');
      return res.json();
    })
    .then(data => setUsers(data))
    .catch(err => setError(err.message));
}, []);
```

Using async/await

```
useEffect(() => {
  const fetchUsers = async () => {
    try {
      setLoading(true);
      const res = await fetch('/api/users');
      if (!res.ok) throw new Error('Failed');
      const data = await res.json();
      setUsers(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };
  fetchUsers();
}, []);
```

Using Axios

```
npm install axios
```

```
// services/api.js
import axios from 'axios';
```

```
const api = axios.create({
  baseURL: 'http://localhost:8080/api',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' }
});
```

```
// Add JWT to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```
// Handle global errors
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```



```
);  
  
export default api;
```

CRUD Operations

```
// GET - fetch all  
const getUsers = () => api.get('/users');  
  
// GET - fetch one  
const getUser = (id) => api.get(`/users/${id}`);  
  
// POST - create  
const createUser = (data) => api.post('/users', data);  
  
// PUT - update  
const updateUser = (id, data) => api.put(`/users/${id}`, data);  
  
// DELETE  
const deleteUser = (id) => api.delete(`/users/${id}`);  
  
// Usage in component  
async function handleSubmit(formData) {  
  try {  
    const { data } = await createUser(formData);  
    console.log('Created:', data);  
  } catch (err) {  
    console.error(err.response?.data?.message);  
  }  
}
```

Fetch vs Axios

Feature	Fetch	Axios
Built-in	Yes	No (install needed)
Auto JSON parsing	No (.json() needed)	Yes
Request timeout	Manual (AbortController)	Built-in
Interceptors	No	Yes
Browser support	Modern only	Wider
Request body	Stringify manually	Auto

18. Error Boundaries

Error boundaries are class components that catch JavaScript errors in their child component tree, log them, and display a fallback UI. They DO NOT catch errors in event handlers, async code, or in the boundary itself.

Creating an Error Boundary

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  // Updates state when an error occurs
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // Logs error info (e.g., send to monitoring service)
  componentDidCatch(error, errorInfo) {
    console.error('Caught:', error, errorInfo);
    // logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div>
          <h2>Something went wrong.</h2>
          <p>{this.state.error?.message}</p>
          <button onClick={() => this.setState({hasError: false})}>
            Try Again
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

What Error Boundaries Don't Catch

- Event handlers (use try/catch inside them)
- Asynchronous code (setTimeout, promises)
- Server-side rendering
- Errors thrown in the error boundary itself

Note: As of 2025, there's no functional component equivalent. Libraries like react-error-boundary provide a hook-friendly wrapper around class-based boundaries.

19. Higher-Order Components & Render Props

Higher-Order Component (HOC)

A HOC is a function that takes a component and returns a new component with enhanced behavior.
Pattern: `const Enhanced = withSomething(WrappedComponent)`

```
// HOC that adds loading state
function withLoading(Component) {
  return function WithLoading({ isLoading, ...props }) {
    if (isLoading) return <p>Loading...</p>;
    return <Component {...props} />;
  };
}

// Usage
const UserListWithLoading = withLoading(UserList);
<UserListWithLoading isLoading={loading} users={users} />
```

Authentication HOC

```
function withAuth(Component) {
  return function ProtectedComponent(props) {
    const { user } = useAuth();
    if (!user) return <Navigate to="/login" />;
    return <Component {...props} user={user} />;
  };
}

const ProtectedDashboard = withAuth(Dashboard);
```

Render Props

A technique where a component takes a function as a prop that returns React elements, letting parent control rendering.

```
class MouseTracker extends React.Component {
  state = { x: 0, y: 0 };

  handleMouseMove = (e) => {
    this.setState({ x: e.clientX, y: e.clientY });
  };

  render() {
    return (
      <div onMouseMove={this.handleMouseMove}>
        {this.props.render(this.state)}
      </div>
    );
  }
}

// Usage
<MouseTracker render={({x, y}) => (
  <h1>Mouse at ({x}, {y})</h1>
)} />
```

Note: Modern React favors **custom hooks** over HOCs and render props for sharing stateful logic — they're cleaner and avoid 'wrapper hell'.

20. Performance Optimization

React.memo

Memoizes a component to prevent unnecessary re-renders when props haven't changed.

```
const ExpensiveChild = React.memo(function ExpensiveChild({ value }) {
  console.log('Rendering...');
  return <div>{value}</div>;
});

// With custom comparison function
const Component = React.memo(MyComp, (prev, next) => {
  return prev.id === next.id; // skip re-render if id same
});
```

useMemo & useCallback

Cache values and functions to avoid recomputation or unnecessary child re-renders.

```
// Memoize expensive calculation
const sortedList = useMemo(
  () => items.sort((a,b) => a.name.localeCompare(b.name)),
  [items]
);

// Memoize callback so child (wrapped in memo) doesn't re-render
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

Code Splitting with React.lazy

Load components only when needed to reduce initial bundle size.

```
import { lazy, Suspense } from 'react';

const HeavyComponent = lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback=<div>Loading...</div>>
      <HeavyComponent />
    </Suspense>
  );
}

// With React Router
<Route path="/admin" element={
  <Suspense fallback=<Spinner />>
    <AdminPanel />
  </Suspense>
} />
```

Other Performance Tips

- Use the **production build** in deployment (drops dev warnings)
- Virtualize long lists with react-window or react-virtuoso
- Avoid inline object/function creation in props when child is memoized

- Use stable, unique key values in lists
- Lift state up only as high as necessary — keep state local when possible
- Use `useTransition` for non-urgent state updates (React 18+)
- Profile with React DevTools Profiler to find bottlenecks

21. Testing React Apps

Two popular tools: **Jest** (test runner + assertion library) and **React Testing Library (RTL)** for rendering and querying components.

Setup

```
npm install --save-dev @testing-library/react @testing-library/jest-dom jest
```

Basic Component Test

```
// Greeting.jsx
export default function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Greeting.test.jsx
import { render, screen } from '@testing-library/react';
import Greeting from './Greeting';

test('renders greeting with name', () => {
  render(<Greeting name="Alice" />);
  expect(screen.getByText('Hello, Alice!')).toBeInTheDocument();
});
```

Testing User Interactions

```
import { render, screen, fireEvent } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

test('increments count on click', async () => {
  const user = userEvent.setup();
  render(<Counter />);

  expect(screen.getByText('Count: 0')).toBeInTheDocument();
  await user.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

Testing Async Code

```
test('fetches and displays users', async () => {
  render(<UserList />);
  expect(screen.getByText(/loading/i)).toBeInTheDocument();

  // waitFor waits until assertion passes
  await waitFor(() => {
    expect(screen.getByText('Alice')).toBeInTheDocument();
  });
});
```

Mocking API Calls

```
import axios from 'axios';
jest.mock('axios');

test('loads users', async () => {
  axios.get.mockResolvedValue({
    data: [{ id: 1, name: 'Alice' }]
  });
});
```

```
render(<UserList />);  
expect(await screen.findByText('Alice')).toBeInTheDocument();  
});
```

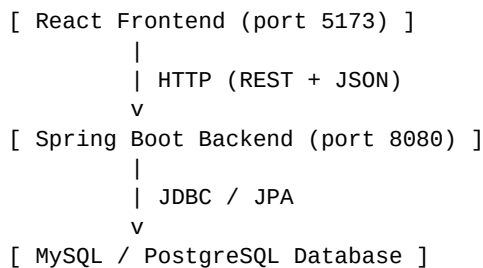
Common RTL Queries

- `getByText` — find by text content
- `getByRole` — find by ARIA role (preferred)
- `getByLabelText` — find form inputs by label
- `getByPlaceholderText` — find input by placeholder
- `getByTestId` — find by data-testid (last resort)

22. Integrating React with Spring Boot

In a Java full stack role, React is typically the frontend and Spring Boot serves REST APIs. This section covers how to connect them.

Architecture Overview



Spring Boot REST Controller

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "http://localhost:5173")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public User createUser(@RequestBody @Valid UserDto dto) {
        return userService.create(dto);
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id, @RequestBody UserDto dto) {
        return userService.update(id, dto);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

React Side - Calling the API

```
// services/userService.js
import axios from 'axios';

const API_URL = 'http://localhost:8080/api/users';
```



```

export const userService = {
  getAll: () => axios.get(API_URL),
  getById: (id) => axios.get(`${API_URL}/${id}`),
  create: (user) => axios.post(API_URL, user),
  update: (id, user) => axios.put(`${API_URL}/${id}`, user),
  delete: (id) => axios.delete(`${API_URL}/${id}`)
};

// UserList.jsx
function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    userService.getAll().then(res => setUsers(res.data));
  }, []);

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Handling CORS

CORS errors occur when frontend and backend are on different origins. Two approaches:

- **Per-controller:** `@CrossOrigin(origins = "http://localhost:5173")`
- **Global config (recommended):**

```

@Configuration
public class WebConfig implements WebMvcConfigurer {
  @Override
  public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
      .allowedOrigins("http://localhost:5173")
      .allowedMethods("GET", "POST", "PUT", "DELETE")
      .allowedHeaders("*")
      .allowCredentials(true);
  }
}

```

JWT Authentication Flow

```

// 1. Login (React)
const login = async (credentials) => {
  const res = await axios.post('/api/auth/login', credentials);
  localStorage.setItem('token', res.data.token);
};

// 2. Send token on every request (axios interceptor)
axios.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// 3. Spring Boot - JWT filter validates token on each request
// (Spring Security with JwtAuthenticationFilter)

```

Building React for Production with Spring Boot

Two deployment models:

- **Separate deployments:** React on Nginx/CDN, Spring Boot on Tomcat — independent scaling
- **Bundled:** Build React, copy dist/ to Spring Boot's src/main/resources/static, serve as single JAR

```
# Build React
cd frontend
npm run build

# Copy build to Spring Boot static folder
cp -r dist/* ../backend/src/main/resources/static/

# Build Spring Boot - serves React + API as single JAR
cd ../backend
./mvnw clean package
java -jar target/app.jar
```

23. Best Practices & Folder Structure

Recommended Folder Structure

```
src/
├── assets/           # Images, fonts, static files
├── components/       # Reusable components
│   ├── common/      # Buttons, Inputs, Modal, etc.
│   ├── layout/      # Header, Footer, Sidebar
│   └── pages/        # Page-level components (route targets)
│       ├── Home/
│       ├── Login/
│       └── Dashboard/
├── hooks/           # Custom hooks (useAuth, useFetch)
├── context/         # React Context providers
├── services/        # API calls (userService.js, authService.js)
├── store/           # Redux store, slices
├── utils/           # Helper functions, constants
├── styles/          # Global CSS, themes
├── App.jsx          # Root component
└── main.jsx         # Entry point
```

Coding Best Practices

- Keep components **small and focused** (single responsibility)
- Use **functional components** with hooks over class components
- Use **destructuring** for props and state
- Use **meaningful component & variable names**
- Always handle **loading and error states** when fetching data
- Keep components **pure** — same input always produces same output
- Use **fragments** instead of wrapping divs unnecessarily
- Extract repeated logic into **custom hooks**
- Move API calls to a **services** layer (not inside components)
- Avoid deeply nested ternaries — use early returns
- Use **environment variables** for API URLs (VITE_API_URL)
- Add **PropTypes** or use **TypeScript** for type safety
- Write **tests** for critical business logic
- Use **ESLint + Prettier** for code consistency

Common Anti-Patterns to Avoid

- Mutating state directly (use setters / immutable updates)
- Using array index as key for dynamic lists
- Putting business logic inside JSX
- Not cleaning up effects (timers, subscriptions)
- Calling hooks conditionally
- Overusing useEffect for derived values (use computed values instead)
- Prop drilling 4+ levels deep (use Context or state management)

24. L1 Interview Questions (50+)

Level 1 interviews focus on **fundamentals**: core concepts, syntax, and basic understanding. These questions are commonly asked for Associate Software Engineer / fresher / 0-2 year experience roles.

Q1. What is React?

React is an open-source JavaScript library developed by Facebook (Meta) for building user interfaces, particularly single-page applications. It uses a component-based architecture and a virtual DOM for efficient updates.

Q2. Is React a library or a framework?

React is technically a **library**, not a framework. It focuses only on the view layer (UI). For features like routing or state management, you need additional libraries like React Router or Redux.

Q3. What are the main features of React?

Component-based architecture, Virtual DOM, JSX syntax, unidirectional data flow, declarative UI, reusable components, and a rich ecosystem of supporting libraries.

Q4. What is JSX?

JSX (JavaScript XML) is a syntax extension that allows writing HTML-like code inside JavaScript. It gets transpiled by Babel into `React.createElement()` calls.

Q5. Why can't browsers read JSX?

Browsers can only understand standard JavaScript. JSX is not valid JavaScript, so it must be transpiled by tools like Babel into regular JavaScript before being sent to the browser.

Q6. What is the Virtual DOM?

The Virtual DOM is an in-memory JavaScript representation of the real DOM. When state changes, React creates a new virtual DOM tree, compares it with the previous one (diffing), and updates only the changed parts in the real DOM (reconciliation). This is much faster than manipulating the real DOM directly.

Q7. What is the difference between Real DOM and Virtual DOM?

Real DOM: Updates slowly, manipulates HTML directly, can update HTML directly, expensive operations.

Virtual DOM: Updates faster, is a lightweight copy in memory, cannot directly update HTML, cheap operations.

Q8. What is a component in React?

A component is an independent, reusable piece of UI that can be a function or a class. Components return JSX describing what should appear on the screen and can manage their own state and lifecycle.

Q9. What are the types of components in React?

Two types: **Functional components** (JavaScript functions, modern approach using hooks) and **Class components** (ES6 classes extending `React.Component`, legacy approach with lifecycle methods).

Q10. What are props?

Props (properties) are read-only inputs passed from a parent component to a child component. They allow components to be reusable with different data. Props cannot be modified by the receiving component.

Q11. What is state in React?

State is an object that stores data which can change over time within a component. Unlike props, state is managed internally and triggers a re-render when updated using a setter (like `setState` or the `useState` hook setter).

Q12. What is the difference between state and props?

Props: Read-only, passed from parent, immutable inside child, used for configuration.

State: Managed within component, mutable via setter, triggers re-renders, used for internal data.

Q13. What are hooks in React?

Hooks are functions introduced in React 16.8 that let functional components use state and other React features (lifecycle, context, refs) without writing classes. Examples: `useState`, `useEffect`, `useContext`, `useRef`, `useMemo`, `useCallback`.

Q14. What are the rules of hooks?

1. Only call hooks at the top level of a function — never inside loops, conditions, or nested functions. 2. Only call hooks from React function components or other custom hooks. 3. Hook names must start with 'use'.

Q15. What does `useState` return?

`useState` returns an array with two elements: the current state value and a setter function. We typically destructure: `const [count, setCount] = useState(0)`.

Q16. What is `useEffect`?

`useEffect` is a hook that lets you perform side effects in functional components, such as data fetching, subscriptions, or manually changing the DOM. It runs after the render is committed to the screen. It can also return a cleanup function.

Q17. What are the differences between `useState` and `useEffect`?

`useState` manages component state and triggers re-renders. **`useEffect`** runs side effects after render, not during render. `useState` returns `[value, setter]`; `useEffect` takes `(callback, dependencies)`.

Q18. How do you pass data from child to parent?

By passing a callback function from parent to child as a prop. The child calls this function with the data, and the parent receives it and can update its own state accordingly.

Q19. What is prop drilling?

Prop drilling is passing props through multiple intermediate components that don't actually need the data, just to get it to a deeply nested component. It can make code harder to maintain. Solutions: Context API, Redux, or component composition.

Q20. What is the Context API?

Context API provides a way to share values (like themes, user info, language) between components without explicitly passing props through every level. It's used for global data that many components need access to.

Q21. What is a key prop and why is it important?

The key prop is a unique identifier React uses when rendering lists to track which items have changed, been added, or removed. Keys help React efficiently update the DOM. They must be unique among siblings and stable across renders.

Q22. Why shouldn't we use array index as key?

Using array index as key can cause bugs when items are reordered, added, or removed because React will associate state with the wrong items, leading to incorrect rendering and performance issues. Use a stable unique ID instead.

Q23. What is conditional rendering?

Rendering different UI based on conditions using JavaScript operators like if/else, ternary `?:`, logical AND `&&`, or by returning null to render nothing.

Q24. What is the difference between controlled and uncontrolled components?

Controlled: Form data is handled by React state (single source of truth). **Uncontrolled:** Form data is handled by the DOM itself, accessed via refs. Controlled components are preferred.

Q25. What is the purpose of the `render()` method?

In class components, `render()` is a required method that returns the JSX to be displayed. It must be a pure function — same input produces same output, no side effects.

Q26. What is React Fragment?

Fragment is a way to group multiple elements without adding extra nodes to the DOM. Useful when a component must return multiple elements but you don't want a wrapper div. Syntax: `<>...</>` or `<React.Fragment>`.

Q27. What is the difference between class and functional components?

Functional: simpler syntax, use hooks for state/lifecycle, no 'this' keyword, modern standard. Class: ES6 class syntax, use `this.state` and lifecycle methods, requires 'this' binding, legacy.

Q28. How do you handle events in React?

Events are handled using camelCase event handlers like `onClick`, `onChange`. Pass a function reference (not a string) as the handler. React wraps native events in `SyntheticEvent` for cross-browser compatibility.

Q29. What is the difference between `createElement` and `cloneElement`?

`createElement` creates a new React element from scratch. `cloneElement` creates a copy of an existing element, optionally with new props or children. `cloneElement` is useful in HOCs.

Q30. What is reconciliation?

Reconciliation is the process by which React updates the DOM. When state changes, React creates a new virtual DOM tree, compares it with the previous one using a diffing algorithm, and applies only the minimal necessary changes to the real DOM.

Q31. What is the difference between `useEffect` and `useLayoutEffect`?

`useEffect` runs asynchronously after the browser paints. **`useLayoutEffect`** runs synchronously after DOM mutations but before the browser paints. Use `useLayoutEffect` when you need to measure DOM nodes or prevent visual flicker.

Q32. What are synthetic events in React?

`SyntheticEvent` is React's cross-browser wrapper around the browser's native event. It has the same interface as native events but works consistently across all browsers. Examples: `SyntheticMouseEvent`, `SyntheticKeyboardEvent`.

Q33. How do you create a list in React?

Use JavaScript's `.map()` method to transform an array into JSX elements. Each element must have a unique key prop: `items.map(item => <li key={item.id}>{item.name}</li>)`.

Q34. What is the default behavior of `useEffect` without dependency array?

Without a dependency array, `useEffect` runs after EVERY render. With an empty array `[]`, it runs only once (on mount). With `[dep1, dep2]`, it runs when any listed dependency changes.

Q35. How do you optimize performance in React?

Use `React.memo` for component memoization, `useMemo` for expensive computations, `useCallback` for stable function references, code splitting with `React.lazy` + `Suspense`, virtualize long lists, use stable keys, and avoid inline object/function props for memoized children.

Q36. What is `React.memo`?

`React.memo` is a higher-order component that memoizes a functional component. It re-renders only when its props change, preventing unnecessary re-renders. Similar to `PureComponent` for class components.

Q37. What is `useMemo` used for?

`useMemo` memoizes the return value of an expensive function so it's only recomputed when one of its dependencies changes. Example: `const sorted = useMemo(() => expensiveSort(list), [list])`.

Q38. What is `useCallback` used for?

`useCallback` memoizes a function reference so the same function instance is returned across renders unless dependencies change. Useful when passing callbacks to memoized child components to prevent re-renders.

Q39. What is the difference between `useMemo` and `useCallback`?

`useMemo` memoizes the **result** of a function call. `useCallback` memoizes the **function itself**. `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`.

Q40. How do you fetch data in React?

Inside `useEffect`, call `fetch` or `axios` with the URL, handle the promise (`await` or `.then`), update state with the result, and include cleanup if needed. Manage loading and error states separately.

Q41. Can you update state directly in React?

No. Always use the setter function (`setState` or the `useState` setter). Direct mutation won't trigger a re-render and breaks React's diffing. Example: `setUser({...user, name: 'Alice'})` instead of `user.name = 'Alice'`.

Q42. What is the `children` prop?

`children` is a special prop that automatically contains the content between a component's opening and closing tags. It enables component composition: `<Card><p>Hello</p></Card>`.

Q43. What are refs in React?

Refs provide a way to access DOM nodes or React elements directly. Created with `useRef` hook or `createRef` in class components. Common uses: focusing inputs, managing animations, integrating with third-party DOM libraries.

Q44. What is the difference between `useRef` and `useState`?

useState: Triggers re-render when value changes, used for UI state. **useRef:** Does NOT trigger re-render when changed, used to persist values across renders or to access DOM elements.

Q45. How does React Router work?

React Router enables client-side routing in SPAs. It uses BrowserRouter (HTML5 history API) or HashRouter, defines routes with <Route>, navigates with <Link> or useNavigate, and matches URL paths to components without full page reloads.

Q46. What is the difference between Link and anchor tag?

<a> causes a full page reload. <Link> from React Router uses the History API to change URL without reloading, preserving React state and providing a smooth SPA experience.

Q47. What is a higher-order component (HOC)?

An HOC is a function that takes a component and returns a new component with enhanced functionality. Pattern: `const Enhanced = withSomething(Component)`. Examples: `withAuth`, `withLoading`. Modern React prefers custom hooks for this pattern.

Q48. What is the purpose of useReducer?

useReducer is an alternative to useState for managing complex state logic. It takes a reducer function and initial state, returns [state, dispatch]. Better than useState when state has multiple sub-values or next state depends on previous state.

Q49. What is the difference between Redux and Context API?

Context: Built into React, good for low-frequency global data (theme, auth). **Redux:** External library, better for complex state with frequent updates, has dev tools, middleware support, predictable state management.

Q50. How do you handle forms in React?

Use controlled components: bind input values to state, update state on onChange. For complex forms, use libraries like React Hook Form or Formik to handle validation and submission more easily.

Q51. What is StrictMode in React?

<React.StrictMode> is a development-only wrapper that highlights potential problems: detecting unsafe lifecycle methods, warning about legacy API usage, identifying side effects in render (by intentionally double-invoking certain functions in dev).

Q52. What is the difference between componentDidMount and useEffect?

componentDidMount runs once after mount in class components. `useEffect(() => {}, [])` with empty deps is the functional equivalent. `useEffect` can also handle update and unmount logic via the dependency array and cleanup function.

Q53. How do you pass data to a React component?

Via props from parent: `<Child name="Alice" age={25} />`. The child accesses them: `function Child({ name, age }) { ... }`. For deeply nested components, use Context API or state management library.

Q54. What is React DevTools?

React DevTools is a browser extension (Chrome, Firefox) that lets you inspect React component trees, view props/state in real time, profile performance, and identify why components re-rendered. Essential for debugging.

Q55. What are some common React lifecycle methods?

Class components: **constructor** (init state), **render** (return JSX), **componentDidMount** (after first render), **componentDidUpdate** (after re-render), **componentWillUnmount** (before removal). Functional equivalents are handled with `useEffect`.

25. L2 Interview Questions (50+)

Level 2 interviews focus on **deeper understanding**: internals, design patterns, performance, edge cases, and real-world scenarios. Expected for Associate SE roles with 1-3 years of experience or strong fundamentals.

Q1. Explain the reconciliation algorithm in React.

React uses a heuristic $O(n)$ diffing algorithm to compare the new virtual DOM tree with the previous one. Key assumptions: (1) Two elements of different types produce different trees (React tears down old tree and builds new). (2) Lists of children with stable keys can be matched efficiently. The algorithm walks both trees in parallel, comparing root elements first, then recursively handling children. Only the minimal set of changes is applied to the real DOM.

Q2. What is the React Fiber architecture?

Fiber is the reimplementation of React's core reconciliation algorithm introduced in React 16. Key benefits: incremental rendering (work can be split into chunks and spread over multiple frames), ability to pause/resume/reuse work, prioritization of updates (urgent vs non-urgent), and better handling of errors via error boundaries. Each component has a 'fiber' node representing a unit of work.

Q3. How does React batch state updates?

React groups multiple state updates that occur in the same event handler or async callback into a single re-render for performance. In React 18+, automatic batching extends to promises, `setTimeouts`, and native event handlers — not just React event handlers. To opt out, use `flushSync()`.

Q4. What happens when you call `setState` multiple times in a row?

If updates are independent, only the last value 'wins' because each setter call uses the current closure's state. To compound updates, use the functional form: `setCount(prev => prev + 1)`. React batches these and applies them sequentially, ensuring each sees the latest state.

Q5. Explain the difference between Virtual DOM, Shadow DOM, and Real DOM.

Real DOM: Browser's document object model — expensive to update directly. **Virtual DOM**: Lightweight JavaScript representation used by React for efficient diffing. **Shadow DOM**: A browser feature for encapsulation in Web Components — isolates styles and DOM (different concept entirely).

Q6. What is the difference between Pure Component and `React.memo`?

PureComponent (class) does a shallow comparison of props and state to decide whether to re-render. **React.memo** (functional) is equivalent for functional components but compares only props by default — you can pass a custom comparison function as the second argument.

Q7. How does React handle errors in components?

Errors during rendering, in lifecycle methods, and in constructors of child components are caught by **Error Boundaries**. An error boundary is a class component implementing static `getDerivedStateFromError` and/or `componentDidCatch`. Errors in event handlers, async code, or in the boundary itself are NOT caught — use `try/catch` for those.

Q8. Explain `useEffect` cleanup. When is it called?

The cleanup function returned from `useEffect` runs in two scenarios: (1) Before the effect runs again due to dependency change, (2) When the component unmounts. Used to clear timers, unsubscribe from events, cancel API requests, etc. Critical for preventing memory leaks.

Q9. What is the difference between `useEffect`'s cleanup and `componentWillUnmount`?

`componentWillUnmount` runs only once when the component unmounts. `useEffect`'s cleanup runs both before re-running the effect (when deps change) AND when unmounting. This makes `useEffect` cleanup more flexible but requires careful dependency management.

Q10. How would you optimize a slow React app?

Steps: 1) Profile with React DevTools to identify bottlenecks. 2) Use `React.memo` / `useMemo` / `useCallback` strategically. 3) Code-split with `React.lazy` and `Suspense`. 4) Virtualize long lists (`react-window`). 5) Avoid unnecessary state — use refs for non-rendering data. 6) Debounce/throttle expensive operations. 7) Use production build. 8) Optimize images and bundle size. 9) Move heavy work to Web Workers.

Q11. What is the difference between `useCallback` and `useMemo` with real-world example?

`useCallback(fn, deps)` returns the same function reference unless deps change. `useMemo(() => fn, deps)` is equivalent. Use `useCallback` when passing handlers to `React.memo`-wrapped children. Use `useMemo` when you have an expensive computation like sorting a large array.

Q12. Explain controlled vs uncontrolled forms with trade-offs.

Controlled: React state is single source of truth — easy validation, conditional logic, but re-renders on every keystroke. **Uncontrolled:** DOM holds the value (via refs) — better performance for large forms, less code, but harder validation. For most apps, controlled is preferred; for very large forms, libraries like React Hook Form use uncontrolled inputs internally for performance.

Q13. How does the `useState` hook work internally?

React maintains an internal linked list of hook states for each component, in the order they're called. On each render, hooks are read in the same order — that's why you can't call hooks conditionally. The setter function tells React to schedule a re-render, and React will return the latest value from its internal store on the next render.

Q14. What is hydration in React?

Hydration is the process where React attaches event listeners to existing server-rendered HTML to make it interactive. Used in SSR (Server-Side Rendering) frameworks like Next.js. The HTML appears instantly (fast initial paint), and React 'hydrates' it by reusing the existing DOM rather than re-creating it.

Q15. What are React Portals?

Portals let you render a component's children into a different DOM node outside the parent's hierarchy. Useful for modals, tooltips, popups that need to escape parent's overflow/z-index constraints. Usage: `ReactDOM.createPortal(child, container)`. Events still bubble up the React tree, not the DOM tree.

Q16. How does Redux differ from `useReducer` + `Context`?

useReducer + Context: Built-in, no library, but Context causes all consumers to re-render when value changes — bad for high-frequency updates. **Redux:** External, has middleware (`redux-thunk`, `redux-saga`), DevTools, selectors with memoization (`reselect`), better for large apps. Redux Toolkit reduces boilerplate significantly.

Q17. What are middleware in Redux?

Middleware sits between dispatching an action and the action reaching the reducer. Lets you intercept actions to add logging, async logic, etc. Examples: `redux-thunk` (allows action creators to return functions for async logic), `redux-saga` (uses generators for complex async flows).

Q18. How would you persist Redux state across page reloads?

Use `redux-persist` library to save state to `localStorage/sessionStorage`. Configure a `persistor` and wrap the `Provider` with `PersistGate`. Alternative: manually subscribe to store changes and write to `localStorage`, hydrate on app start. Be careful not to persist sensitive data.

Q19. What is the difference between server-side and client-side rendering?

CSR: Browser downloads JS bundle, then React renders — fast subsequent navigations, slower first load, bad for SEO. **SSR:** Server renders HTML and sends to browser — fast first paint, good for SEO, but more server load and complex setup. Next.js supports both plus SSG (static generation).

Q20. What is Suspense?

Suspense lets components 'suspend' rendering while waiting for asynchronous data (code, data, images). React shows a fallback UI until the dependency resolves. Used today for code splitting with `React.lazy`; React 18+ enables broader data-fetching use cases.

Q21. Explain `useTransition` and `useDeferredValue`.

`useTransition`: Marks state updates as non-urgent transitions, allowing React to interrupt them with urgent updates (like typing). Returns `[isPending, startTransition]`. **`useDeferredValue`:** Returns a deferred version of a value that lags behind during heavy renders. Both improve perceived performance for expensive renders.

Q22. How do you handle authentication in a React app?

Common pattern with JWT: 1) Login endpoint returns JWT token. 2) Store in `localStorage` or `httpOnly` cookie. 3) Attach token to axios interceptor as Authorization header. 4) Create `AuthContext` with user state. 5) Wrap protected routes with a `ProtectedRoute` component that redirects if no user. 6) Handle token expiry with refresh tokens or auto-logout on 401.

Q23. Where would you store JWT tokens? `localStorage` vs cookies?

`localStorage`: Easy to use, accessible by JavaScript — vulnerable to XSS attacks. **`httpOnly` Cookies:** Not accessible to JavaScript, sent automatically with requests — more secure against XSS but vulnerable to CSRF (mitigate with `SameSite`, CSRF tokens). Best practice: use `httpOnly` cookies with `SameSite=Strict` for security.

Q24. How do you prevent unnecessary re-renders?

1) Use `React.memo` for components that receive same props. 2) Use `useCallback` for callback props passed to memoized children. 3) Use `useMemo` for expensive computations. 4) Don't create new objects/arrays as props in render. 5) Split components to localize state. 6) Use selectors with shallow equality in `Redux`. 7) Avoid spreading props blindly.

Q25. What is the difference between dependent and independent state?

Independent: Pieces of state that change separately should be in separate `useState` calls. **Dependent / related:** Multiple values that always change together belong in one object state or `useReducer`. Don't store derived values in state — compute them during render.

Q26. How do you handle global error handling in React?

1) Wrap app (or sections) with `Error Boundaries` for render errors. 2) Use a global error context to show toast messages. 3) Configure axios response interceptor to catch HTTP errors centrally. 4) Window-level handlers for uncaught errors and unhandled promise rejections. 5) Send errors to a monitoring service like `Sentry`.

Q27. What are some common security issues in React apps?

1) **XSS:** Avoid `dangerouslySetInnerHTML` with user input. 2) **CSRF:** Use anti-CSRF tokens with cookie auth. 3) Don't store sensitive data in `localStorage`. 4) Validate all input on backend, not just frontend. 5) Keep

dependencies updated (audit with `npm audit`). 6) Use HTTPS. 7) Sanitize URLs to prevent javascript: URI attacks.

Q28. How do you implement lazy loading in React?

Use `React.lazy(() => import('./Component'))` combined with `<Suspense>`. The component code is loaded only when needed (e.g., when its route is visited), reducing initial bundle size. Works well at route level with React Router.

Q29. What is the role of webpack/Vite in a React app?

Webpack/Vite are **bundlers**: they transform JSX/TSX into JS, bundle modules into optimized files, handle CSS/images/static assets, enable hot reloading, code splitting, and minification. Vite is faster in development thanks to native ES modules and esbuild; Webpack is more configurable for complex setups.

Q30. How does the React event system differ from native DOM events?

React uses a synthetic event system: events are not attached directly to DOM nodes but to the root via event delegation (in React 17+, attached to the React root instead of document). React pools events for performance and provides a consistent SyntheticEvent API across browsers.

Q31. How would you test a component that fetches data?

1) Mock the API call (e.g., `jest.mock('axios')`). 2) Render the component with React Testing Library. 3) Use `findBy*` queries which return promises that wait for elements to appear. 4) Assert loading state appears initially. 5) Assert data renders after mock resolves. 6) Test error states by mocking rejection.

Q32. What is the difference between mount and render in testing?

Terms come from Enzyme (largely replaced): **shallow** renders only the component itself without children, **mount** renders full DOM with children. React Testing Library uses just **render** which mounts fully — it encourages testing the component as users see it.

Q33. How would you implement infinite scrolling in React?

Use `IntersectionObserver` to detect when a sentinel element near the bottom of the list enters the viewport, then fetch the next page. Append results to existing state. Combine with a 'loading more' indicator. Alternative: use libraries like `react-infinite-scroll-component` or virtualization libraries like `react-window-infinite-loader`.

Q34. Explain code splitting strategies in React.

1) **Route-based**: Lazy-load each page component — most common. 2) **Component-based**: Lazy-load heavy components that aren't always shown. 3) **Library splitting**: Separate vendor chunks (already done by bundlers). 4) **Dynamic imports based on user action**: e.g., load chart library only when user clicks 'Show analytics'.

Q35. What happens if you forget the dependency array in `useEffect`?

The effect runs after EVERY render. If the effect updates state, it causes infinite re-renders. Always specify the dependency array — empty `[]` for mount-only, or with the dependencies the effect uses.

Q36. How do you handle race conditions in API calls inside `useEffect`?

Use `AbortController` to cancel in-flight requests when dependencies change: `const controller = new AbortController(); fetch(url, {signal: controller.signal}); return () => controller.abort();`. Or use a flag: declare a local `let cancelled = false`, check it before setting state, set to true in cleanup.

Q37. What is the difference between named and default exports in React?

Default export: export default Component — imported with any name: import X from './x'. One per file.
Named export: export const Component = ... — imported with exact name: import { Component } from './x'. Multiple per file. Named exports support better refactoring and tree-shaking.

Q38. How would you architect a large React application?

1) Feature-based folder structure (group by domain, not by file type). 2) Clear separation: components, hooks, services, store, utils. 3) Co-locate tests with code. 4) Use TypeScript for type safety. 5) Centralize API layer (services). 6) Lazy load routes. 7) Establish coding standards (ESLint, Prettier). 8) Use a UI library or design system for consistency. 9) Document with Storybook for reusable components.

Q39. What is the strategy pattern of state management in React?

Choose state management based on need: **Local state (useState):** component-specific UI state. **Lifted state:** shared between siblings. **Context:** low-frequency global data (theme, auth). **Redux / Zustand / Jotai:** complex global state with frequent updates. **Server state libraries (React Query, SWR):** for caching API data.

Q40. Explain React Query / TanStack Query.

A library for managing server state in React. Handles caching, background refetching, deduplication, pagination, optimistic updates, and stale-while-revalidate patterns. Removes the need to manually manage loading/error states or store API data in Redux. Example: `const {data, isLoading} = useQuery({queryKey: ['users'], queryFn: getUsers})`.

Q41. How does Concurrent Mode work in React 18?

Concurrent rendering lets React work on multiple state updates simultaneously, pause work, and prioritize. Key features: automatic batching, `useTransition` (mark updates as non-urgent), `useDeferredValue` (defer expensive re-renders), `Suspense` for data fetching. Enables a more responsive UI.

Q42. What is the difference between Webpack and Vite?

Webpack: Bundles everything upfront, slower cold start in dev, mature and configurable. **Vite:** Serves files via native ES modules in dev (no bundling), uses esbuild for transforms — much faster startup and HMR. For production, Vite uses Rollup for the bundle. Vite is the modern default for new React projects.

Q43. How do you handle environment-specific configurations?

Use environment variables in `.env` files. Vite uses `VITE_*` prefix, CRA uses `REACT_APP_*`. Access via `import.meta.env.VITE_API_URL` (Vite) or `process.env.REACT_APP_API_URL` (CRA). Keep different `.env` files: `.env.development`, `.env.production`. Never commit secrets.

Q44. How do you debug a React app?

1) React DevTools — inspect tree, props, state, profile renders. 2) Browser DevTools — debugger, breakpoints, network tab. 3) `console.log` liberally. 4) Use the Profiler component to measure render times. 5) Use `why-did-you-render` library to find unnecessary re-renders. 6) Source maps in dev for readable stack traces.

Q45. What are some accessibility (a11y) best practices in React?

1) Use semantic HTML (button, nav, main). 2) Provide alt text for images. 3) Use proper labels for form inputs (htmlFor). 4) Maintain keyboard navigation (tabIndex, focus management). 5) Use ARIA attributes where semantic HTML isn't enough. 6) Test with screen readers. 7) Ensure sufficient color contrast. 8) Use `eslint-plugin-jsx-a11y`.

Q46. How would you migrate a class component to functional with hooks?

1) Convert class to function. 2) Replace `this.state` with `useState` calls (one per piece of state). 3) Replace `componentDidMount` with `useEffect` with empty deps. 4) Replace `componentDidUpdate` with `useEffect` with appropriate deps. 5) Replace `componentWillUnmount` logic with `cleanup` return in `useEffect`. 6) Replace `this.method` bindings with regular functions or `useCallback`. 7) Remove all `'this.'` references.

Q47. What is the difference between Next.js and React?

React: The core UI library. **Next.js:** A full-stack framework built on React, providing routing, SSR/SSG, API routes, image optimization, performance optimizations out of the box. For a fast SEO-friendly app, Next.js is the standard choice.

Q48. How would you implement undo/redo functionality?

Maintain an array of past states (history). Each state change pushes the current state to past and clears the future array. Undo: pop from past, push current to future. Redo: pop from future, push current to past. Use `useReducer` for clean implementation with action types: `PERFORM_ACTION`, `UNDO`, `REDO`.

Q49. How does React handle keys differently for siblings vs cousins?

Keys only need to be unique among **siblings** (children of the same parent), not globally unique. Two cousin lists can have the same keys without conflict. React tracks elements within their parent's children array, so keys reset their scope at each parent.

Q50. How do you handle file uploads in React?

Use `<input type="file">` (uncontrolled with `ref` or controlled with `onChange` to capture File object). Wrap files in `FormData`: `const fd = new FormData(); fd.append('file', file);`. Send via `axios/fetch` with `Content-Type: multipart/form-data` (`axios` sets this automatically when body is `FormData`). Show progress via `onUploadProgress`.

Q51. What are some patterns for handling complex component composition?

Compound components: Parent shares state via `Context` with children (e.g., `Tabs/TabList/TabPanel`). **Render props:** Pass a function that returns JSX. **Children as functions:** Variant of render props. **Slot pattern:** Use named children props (header, footer). Choose based on flexibility needed and team familiarity.

Q52. How do you implement search with debouncing?

Create a `useDebounce` custom hook that returns a delayed value. Bind input to local state, use the debounced value as the effect dependency for the actual API call. This prevents firing requests on every keystroke and reduces server load. Typical delay: 300-500ms.

Q53. Explain the concept of pure components.

A pure component renders the same output for the same props and state — no side effects in render, no mutations. `React.memo` and `PureComponent` rely on this purity to skip re-renders via shallow comparison. Pure components are easier to test, reason about, and optimize.

26. Coding Round Problems

Practice these common React coding problems. Try implementing them yourself first, then check the solution.

Problem 1: Counter Component

Build a counter with increment, decrement, and reset buttons.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <button onClick={() => setCount(c => c - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Problem 2: Todo List

Build a todo list with add, toggle, and delete functionality.

```
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  const add = () => {
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text, done: false }]);
    setText('');
  };

  const toggle = (id) => setTodos(todos.map(t =>
    t.id === id ? { ...t, done: !t.done } : t));
  const remove = (id) => setTodos(todos.filter(t => t.id !== id));

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button onClick={add}>Add</button>
      <ul>
        {todos.map(t => (
          <li key={t.id}>
            <input type="checkbox" checked={t.done}
              onChange={() => toggle(t.id)} />
            <span style={{textDecoration: t.done ? 'line-through' : 'none'}}>
              {t.text}
            </span>
            <button onClick={() => remove(t.id)}>X</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Problem 3: Stopwatch


```

function Stopwatch() {
  const [time, setTime] = useState(0);
  const [running, setRunning] = useState(false);
  const intervalRef = useRef(null);

  useEffect(() => {
    if (running) {
      intervalRef.current = setInterval(() => {
        setTime(prev => prev + 10);
      }, 10);
    }
    return () => clearInterval(intervalRef.current);
  }, [running]);

  const format = (ms) => {
    const m = String(Math.floor(ms/60000)).padStart(2, '0');
    const s = String(Math.floor((ms%60000)/1000)).padStart(2, '0');
    const cs = String(Math.floor((ms%1000)/10)).padStart(2, '0');
    return `${m}:${s}.${cs}`;
  };

  return (
    <div>
      <h1>{format(time)}</h1>
      <button onClick={() => setRunning(!running)}>
        {running ? 'Pause' : 'Start'}
      </button>
      <button onClick={() => { setRunning(false); setTime(0); }}>Reset</button>
    </div>
  );
}

```

Problem 4: Search Filter

```

function SearchableList({ items }) {
  const [query, setQuery] = useState('');
  const filtered = useMemo(
    () => items.filter(item =>
      item.toLowerCase().includes(query.toLowerCase())
    ),
    [items, query]
  );
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)}
        placeholder="Search..." />
      <ul>{filtered.map((it, i) => <li key={i}>{it}</li>)}</ul>
    </div>
  );
}

```

Problem 5: Fetch & Display Users

```

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(res => {
        if (!res.ok) throw new Error('Failed');
        return res.json();
      })
  });
}

```



```

        .then(setUsers)
        .catch(e => setError(e.message))
        .finally(() => setLoading(false));
}, []);

if (loading) return <p>Loading...</p>;
if (error) return <p>Error: {error}</p>;
return (
  <ul>{users.map(u => <li key={u.id}>{u.name} - {u.email}</li>)}</ul>
);
}

```

Problem 6: Form Validation

```

function SignupForm() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});

  const validate = () => {
    const err = {};
    if (!form.email) err.email = 'Required';
    else if (!/\S+@\S+\.\S+/.test(form.email)) err.email = 'Invalid email';
    if (!form.password) err.password = 'Required';
    else if (form.password.length < 6) err.password = 'Min 6 chars';
    return err;
  };

  const submit = (e) => {
    e.preventDefault();
    const err = validate();
    setErrors(err);
    if (Object.keys(err).length === 0) {
      console.log('Submit:', form);
    }
  };

  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  return (
    <form onSubmit={submit}>
      <input name="email" value={form.email} onChange={change}
        placeholder="Email" />
      {errors.email && <span>{errors.email}</span>}
      <input name="password" type="password" value={form.password}
        onChange={change} placeholder="Password" />
      {errors.password && <span>{errors.password}</span>}
      <button>Sign Up</button>
    </form>
  );
}

```

Problem 7: Debounced Search

```

function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

function Search() {
  const [query, setQuery] = useState('');

```

```

const [results, setResults] = useState([]);
const debounced = useDebounce(query, 500);

useEffect(() => {
  if (!debounced) { setResults([]); return; }
  fetch(`/api/search?q=${debounced}`)
    .then(r => r.json()).then(setResults);
}, [debounced]);

return (
  <div>
    <input value={query} onChange={e => setQuery(e.target.value)} />
    <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
  </div>
);
}

```

Problem 8: Modal Component

```

function Modal({ isOpen, onClose, children }) {
  useEffect(() => {
    const onKey = (e) => e.key === 'Escape' && onClose();
    if (isOpen) document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey);
  }, [isOpen, onClose]);

  if (!isOpen) return null;
  return ReactDOM.createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        <button onClick={onClose}>X</button>
        {children}
      </div>
    </div>,
    document.body
  );
}

```

27. Quick Revision Cheat Sheet

Essential Hooks

<code>useState(initial)</code>	-> [value, setValue]
<code>useEffect(fn, [deps])</code>	-> side effects + cleanup
<code>useContext(Context)</code>	-> consume context value
<code>useReducer(reducer, init)</code>	-> [state, dispatch]
<code>useRef(initial)</code>	-> mutable ref { current }
<code>useMemo(() => val, [deps])</code>	-> memoized value
<code>useCallback(fn, [deps])</code>	-> memoized function
<code>useLayoutEffect(fn, [deps])</code>	-> sync after DOM mutation
<code>useId()</code>	-> unique stable ID
<code>useTransition()</code>	-> [isPending, startTransition]
<code>useDeferredValue(val)</code>	-> deferred value

Common Patterns

```
// Update state immutably
setUser({ ...user, name: 'Alice' });
setItems([...items, newItem]);
setItems(items.filter(i => i.id !== id));
setItems(items.map(i => i.id === id ? {...i, done: true} : i));

// Conditional rendering
{condition && <Component />}
{condition ? <A/> : <B/>}
{loading ? <Spinner/> : data && <List data={data}/>}

// Lists
{items.map(item => <Item key={item.id} {...item} />)}

// Refs to DOM
const ref = useRef(null);
<input ref={ref} />
ref.current.focus();

// Cleanup pattern
useEffect(() => {
  const timer = setTimeout(...);
  return () => clearTimeout(timer);
}, []);
```

Common Pitfalls

- Mutating state directly: `items.push(x)` won't trigger re-render
- Missing dependency array in `useEffect` causes infinite loops
- Using index as key when list is reorderable
- Calling hooks conditionally (inside if/loop)
- Forgetting `e.preventDefault()` on form submit
- Stale closures — referencing old state in async callbacks
- Setting state during render — causes infinite loop
- Not unsubscribing in `useEffect` cleanup

Performance Quick Wins

- Wrap pure children with `React.memo`
- Use `useMemo` for expensive calculations
- Use `useCallback` for callback props to memoized children
- Lazy-load routes with `React.lazy` + `Suspense`
- Use stable keys (id, not index)
- Virtualize long lists (`react-window`)
- Use production build for deployment

Interview Day Tips

- Think aloud — interviewers want to understand your reasoning
- Clarify requirements before coding (edge cases, inputs, outputs)
- Start simple, then optimize — don't prematurely engineer
- Mention trade-offs (e.g., `useState` vs `useReducer`)
- Be honest if you don't know something — explain how you'd find out
- Practice live coding on a whiteboard or shared editor
- Know your projects deeply — be ready to discuss architecture decisions
- Brush up on JavaScript fundamentals (closures, promises, ES6+)

Good luck with your interviews!

Remember: consistent practice beats memorization. Build small projects, read source code, and engage with the React community on GitHub and Stack Overflow. Mastery comes from doing.